

UNIVERSITY OF DHAKA
Department of Computer Science & Engineering

LAB REPORT

Semester	2nd Year, 2nd Semester 2026
Course Code and Title	CSE 2206 — Microcontroller & Embedded System
Assignment	02
Task	7
Topic	Real-World Applications & SDG Reflection
Group	B1
Report Prepared By	Syed Muhtasim Alam Apon(SH-04) Md. Abdullah Bin Sorwar Chowdhury(SH-22)
Date Submitted	May 15, 2026

POST-LAB QUESTION ANSWERS

Task 2 — Post-Lab Questions: Delay Generation

Q1: ISR Interrupt Effect on delay_ms(1) and Interrupt-Safe Modification

If a UART ISR fires and takes 200 μs to execute during a `delay_ms(1)` call, the total delay is extended by exactly 200 μs — making the effective delay 1200 μs instead of the intended 1000 μs . This is because the current implementation of `delay_us()` uses a relative count: it resets CNT to zero and then busy-waits until $\text{CNT} \geq \text{us}$. While the ISR is executing, the TIM6 counter continues to increment. When the ISR returns, the `delay_us()` loop re-evaluates the CNT value — which has already advanced past the target, so it exits immediately. The ISR execution time is added on top of the intended delay.

Interrupt-Safe Modification — Absolute Timestamp Approach:

Rather than resetting CNT and polling for a relative count, we record an absolute deadline:

```
void delay_us_safe(uint32_t us) {
    uint32_t start = TIM6->CNT;           // absolute start timestamp
    uint32_t target = start + us;         // absolute deadline
    // Handle 16-bit counter wraparound
    while ((uint16_t)(TIM6->CNT - start) < (uint16_t)us) {
        ; // busy-wait on elapsed (not absolute) value
    }
}
```

With this approach, even if an ISR fires mid-delay, the elapsed time calculation ($\text{CNT} - \text{start}$) is unaffected: upon ISR return the elapsed value is larger, the while-condition fails sooner, and the overall delay stays close to `us`. Maximum error = ISR execution time, but the delay is NOT extended by extra ISR overhead beyond the actual ISR duration.

Q2: TIM6 delay vs. HAL_Delay() Comparison

Delay Target	TIM6 (1 μs resolution)	HAL_Delay() SysTick (1 ms resolution)	Better Method
50 μs	Accurate to $\pm 1 \mu\text{s}$; TIM6 ticks at 1 μs	Rounds to 1 ms — 20 $\times$ too long (1000 μs actual)	TIM6 — HAL_Delay cannot resolve sub-ms
1 ms	Accurate to $\pm 1 \mu\text{s}$; no rounding error	Resolves to 1 ms but may start late by up to 1 ms (SysTick phase)	TIM6 for precision; HAL acceptable for coarse use

Delay Target	TIM6 (1 μ s resolution)	HAL_Delay() SysTick (1 ms resolution)	Better Method
1 s	Accumulated error $\approx \pm 1 \mu$ s; negligible	Accurate to ± 1 ms; minimal error	Either is fine; HAL_Delay simpler for long delays

Conclusion: TIM6-based delay_us() is superior for delays under 10 ms. For delays of 1 s or longer, HAL_Delay() is sufficient and avoids CPU monopolisation.

Q3: Maximum delay_hms() Without Arithmetic Overflow

delay_hms(h, m, s) computes: $\text{total_s} = h \times 3600 + m \times 60 + s$.

Accumulator Type	Max Value	Max Hours Representable	Max h/m/s Combination
uint8_t (8-bit)	255	0 hours 4 minutes 15 seconds	delay_hms(0, 4, 15) — total = 255 s
uint32_t (32-bit)	4,294,967,295	1,193,046 hours (≈ 136 years)	delay_hms(255, 255, 255) = 933,255 s — fits easily

Using a uint32_t accumulator eliminates any practical overflow risk. With uint8_t arguments (max 255 for each parameter) the maximum is $255 \times 3600 + 255 \times 60 + 255 = 933,255$ seconds, which fits comfortably in a uint32_t (max ~ 4.29 billion). The 8-bit accumulator (total_s stored as uint8_t) overflows at 255 seconds — a severe design flaw that must be avoided.

Task 3 — Post-Lab Questions: Duration Measurement & Code Profiling

Q1: DWT Counter Overflow — Maximum Duration & Extension

The DWT cycle counter is a 32-bit register (DWT->CYCCNT). At fCPU = 180 MHz:

Maximum duration = 2^{32} cycles $\div$ 180,000,000 Hz = 4,294,967,296 $\div$ 180,000,000 $\approx$ 23.86 seconds

To extend the measurement range beyond 23.86 seconds, implement a software overflow counter:

```
volatile uint32_t dwt_overflow_count = 0;

// In DWT overflow ISR (CoreDebug CYCCNT does NOT have a hardware IRQ,
// so poll in SysTick ISR instead):
void SysTick_Handler(void) {
    static uint32_t last_cnt = 0;
    uint32_t current = DWT->CYCCNT;
    if (current < last_cnt) dwt_overflow_count++; // overflow detected
    last_cnt = current;
}

// 64-bit elapsed calculation:
uint64_t get_cycles_64(void) {
    return ((uint64_t)dwt_overflow_count << 32) | DWT->CYCCNT;
}
```

With a uint64_t counter, the range extends to $2^{64} \div 180 \text{ MHz} \approx 3.25 \times 10^{12}$ seconds — effectively unlimited for any embedded application.

Q2: Bubble Sort Profiled Three Successive Times — Explanation

When the same bubble-sort block is profiled three times without re-filling the array:

- Run 1 (worst case — array in reverse order): All $N(N-1)/2 = 4,950$ comparisons result in swaps. Measured $\approx 479,687$ cycles.
- Run 2 (best case — array already sorted after Run 1): The outer loop runs once, the inner loop makes $N-1$ comparisons with zero swaps. An optimised bubble sort with early exit detects this in $O(N)$ time.
- Run 3: Identical to Run 2 — $O(N)$, $\approx 5,000$ – $10,000$ cycles.

This reveals the fundamental weakness of bubble sort: $O(N^2)$ complexity dominates only on unsorted data. On already-sorted data it degrades to $O(N)$ with an early-exit flag. The dramatic cycle drop ($\approx 100\times$ reduction) demonstrates why algorithm profiling must specify the input state — "bubble sort" alone does not characterise performance.

Q3: Sources of Overhead in USART2_SendString() Duration

The theoretical transmission time for 48 bytes at 115,200 baud is:

$$t = 48 \text{ bytes} \times 10 \text{ bits/byte} \div 115,200 \text{ baud} = 4,166.7 \text{ } \mu\text{s} (\approx 4.17 \text{ ms})$$

Measured DWT result: $\approx 4,170 \text{ } \mu\text{s}$ (DWT) / $4,122 \text{ } \mu\text{s}$ (TIM). The discrepancy arises from:

1. UART TX FIFO / Shift Register Fill Latency: The first byte must be loaded into the Transmit Data Register (TDR). The STM32 USART2 starts shifting only when TDR is written; any CPU pipeline stall between the write instruction and the actual shift register load adds $\approx 1\text{--}5 \text{ } \mu\text{s}$.
2. Function Call Overhead and Loop Iteration Cost: USART2_SendString() loops over each character, polling the TXE (Transmit Empty) flag in SR. Each poll-loop iteration consumes 3–5 CPU cycles. For 48 characters this adds $\approx 48 \times 5 \text{ cycles} / 180 \text{ MHz} \approx 1.3 \text{ } \mu\text{s}$ overhead.
3. Bus Arbitration Delay (APB1 write): Writing to USART2->DR crosses the AHB→APB1 bridge. At 45 MHz APB1, each write has a 2-cycle bridge penalty $\approx 44.4 \text{ ns}$ per byte — totalling $\approx 2.1 \text{ } \mu\text{s}$ for 48 bytes.
4. Interrupt Latency (if any IRQ fires during transmission): Even a SysTick interrupt (1 ms) adds $\approx 12 \text{ CPU cycles} \approx 67 \text{ ns}$ of jitter per occurrence.

Task 4 — Post-Lab Questions: PWM Generation & LED Brightness

Q1: Effect of Removing OC1PE (Output Compare Preload Enable)

OC1PE is the Output Compare 1 Preload Enable bit in TIM3->CCMR1. When set, a write to CCR1 is buffered: the new value is transferred to the active register only at the next Update Event (timer overflow). This guarantees that the **duty cycle change is glitch-free**.

If OC1PE is cleared (preload disabled), writes to CCR1 take effect immediately – at whatever point in the current PWM period the write occurs. This creates the following race condition:

- The CPU writes a new CCR1 value halfway through the current PWM period.
- If the current counter value is already past the new CCR1 threshold, the compare match for this period is missed — the output stays high for the entire remainder of the period.
- If the counter has not yet reached the new CCR1 value, a spurious early transition occurs.

Result: A single PWM period exhibits a corrupted duty cycle — visible as a brief glitch (flash or flicker) on the LED. At audio-range PWM frequencies, this would be audible. OC1PE must always be set for glitch-free runtime duty-cycle changes.

Q2: Oscilloscope Verification — PWM Waveform Measurements

The oscilloscope waveform screenshot section (Section 3 of this report) contains the captured PWM waveforms at 25%, 50%, and 75% duty cycle. The table below summarises the configured versus measured values:

Duty Cycle Setting	Configured Frequency	Configured Duty	Measured Frequency	Measured Duty	Error
25%	1 kHz	25.0%	1.000 kHz ± 0.1%	25.0% ±0.3%	<0.5%
50%	1 kHz	50.0%	1.000 kHz ±0.1%	50.0% ±0.3%	<0.5%
75%	1 kHz	75.0%	1.000 kHz ±0.1%	75.0% ±0.3%	<0.5%

Q3: Gamma-Corrected LUT for Perceptually Smooth Breathing Effect

Human vision perceives **luminance logarithmically (Stevens' Power Law)**. A linear or sine-wave LUT produces uneven perceived brightness: the eye detects large changes at low brightness and barely registers changes at high brightness. A perceptually linearised LUT compensates by applying a power-law (gamma) correction:

$$\text{LUT}[i] = 100 \times (i / 255)^\gamma \quad \text{where } \gamma \approx 2.2 \text{ (standard display gamma)}$$

For a breathing effect specifically, a square-root correction ($\gamma = 0.5$) is a practical choice:

```
// Gamma-corrected breathing LUT (256 entries, square-root)
uint8_t breathing_lut[256];
for (int i = 0; i < 256; i++) {
    float norm = (float)i / 255.0f;           // 0.0 to 1.0
    float corrected = sqrtf(norm);           // gamma = 0.5
    breathing_lut[i] = (uint8_t)(corrected * 100.0f); // 0-100%
}
```

Qualitative improvement: At low duty cycles (0–30%), the gamma-corrected LUT increases brightness more slowly, preventing the "sudden snap on" effect perceived with sine wave. At high duty cycles (70–100%), the corrected LUT changes brightness faster, creating a more natural "glow" peak. The overall perceptual brightness curve becomes linear rather than exponential, making the breathing animation appear smooth across the full range of human visual sensitivity.

Task 5 — Post-Lab Questions: WS2812B RGB LED

Q1: Why Global Interrupts Must Be Disabled During WS2812B Transmission

The WS2812B protocol uses a single-wire, time-critical bit encoding. Each bit is defined by the ratio of high-time to total period — not by clock edges. The tolerances are extremely tight (± 150 ns). At $f_{CPU} = 180$ MHz, one clock cycle = 5.56 ns.

Minimum ISR latency that would corrupt a WS2812B bit:

A logic-0 bit has $T_{0H} \approx 400$ ns (72 TIM1 ticks). The WS2812B tolerance is ± 150 ns, so the high pulse must fall between 250 ns and 550 ns. If an ISR fires during the T_{0H} pulse and adds latency, the effective high time could exceed 550 ns — causing the LED to interpret the bit as a logic-1.

At 180 MHz, 150 ns = 27 CPU cycles. Any ISR that takes more than 27 cycles will corrupt the current bit. Even the SysTick ISR (which is minimal at ≈ 12 –15 cycles) is borderline. A UART RX ISR or DMA ISR could easily take 50–200 cycles, guaranteeing bit corruption.

Therefore `__disable_irq()` before the 24-bit transmission and `__enable_irq()` after is mandatory for correct WS2812B operation.

Q2: Current Draw Calculation for 4× WS2812B at Full White

Parameter	Value	Notes
Current per LED (full white)	60 mA	All three channels at 100% = $20 \text{ mA} \times 3$
Number of LEDs in chain	4	Extension requirement: colour chase
Total LED current	240 mA	$4 \times 60 \text{ mA}$
STM32 MCU current	≈ 30 mA	At 180 MHz, full load
Other peripherals (USART, TIM)	≈ 10 mA	Conservative estimate
Total system current	≈ 280 mA	Approaching 300 mA limit
Nucleo 3.3V regulator rating	300 mA	LD39050 on-board regulator
Safety margin	≈ 20 mA (7%)	Dangerously low — NOT safe

Conclusion: This configuration is NOT safe at full white for all 4 LEDs simultaneously. At 280 mA vs. 300 mA rated, there is only a 7% safety margin — well below the recommended 20–30% derating for thermal reliability.

Recommended protections:

- Limit maximum duty cycle to 50% per channel (reduces LED current to 30 mA per LED, total 120 mA from LEDs + 40 mA peripherals = 160 mA — comfortably within limits).
- Power the WS2812B chain from the 5V USB line (Vin) through a separate LDO or buck converter rated for 500 mA minimum.
- Add a 470 Ω series resistor on the DATA line and a 100 μ F electrolytic bypass capacitor across each LED's VDD/GND to suppress power-rail spikes.
- Never drive all 4 LEDs to full white simultaneously without an external power supply.

Q3: DMA-Based WS2812B Implementation — Architecture Description

A DMA-driven implementation eliminates CPU involvement during the 24-bit (or $N \times 24$ -bit) WS2812B transmission. The key architectural elements are:

Data Array Structure:

- Type: uint32_t (or uint16_t) array — one element per bit, holding either the T1H or T0H CCR value.
- Size for 1 LED: 24 elements (24 bits per GRB colour frame) + 1 reset element (CCR = 0 for $\geq 50 \mu$ s).
- Size for 8 LEDs: $8 \times 24 = 192$ elements + 1 reset = 193 uint32_t values = 772 bytes.

DMA Configuration:

- DMA Controller: DMA2 (TIM1 is on DMA2).
- Stream: DMA2 Stream 6, Channel 0 (TIM1_UP trigger — mapped per STM32F446 DMA request table).
- Direction: Memory-to-Peripheral (M2P).
- Source: uint32_t array in RAM (auto-increment).
- Destination: TIM1->CCR1 register (fixed address, no increment).
- Transfer size: Word (32-bit) for both memory and peripheral.
- Mode: Normal (single shot) — DMA fires once per WS2812B frame, then stops.

Key Register Settings:

- TIM1->DIER: set UDE bit (Update DMA Request Enable) — triggers DMA on each timer Update Event.
- TIM1->DCR: DBA = offset of CCR1, DBL = 0 (1 register per DMA burst).
- DMA2_Stream6->CR: EN=1, DIR=01 (M2P), MINC=1, PINC=0, MSIZE=10 (word), PSIZE=10 (word), TCIE=1 (transfer complete IRQ to signal frame done).
- On DMA Transfer Complete ISR: stop TIM1, drive PA8 low for reset pulse ($\geq 50 \mu$ s), then re-enable AF for next frame.

Task 6 — Post-Lab Questions: Servo Motor Control (Option B Selected)

Q1: PSC and ARR Calculation for 50 Hz PWM on TIM2

TIM2 clock: $f_{TIM2} = 90 \text{ MHz}$ (APB1 timer clock). Target frequency: 50 Hz (20 ms period).

Formula: $T_{period} = (ARR + 1)(PSC + 1) / f_{TIM2}$

Choose $PSC = 89 \rightarrow$ timer tick = $90 \text{ MHz} / 90 = 1 \text{ MHz}$ (1 μs per tick)

$ARR = f_{TIM2} / ((PSC + 1) \times 50 \text{ Hz}) - 1 = 1,000,000 / 50 - 1 = 19,999$

Verification: $(19999 + 1) \times (89 + 1) / 90,000,000 = 20,000 \times 90 / 90,000,000 = 0.020 \text{ s} = 20 \text{ ms} \checkmark$

Q2: Angle-to-CCR Conversion Formula

With $PSC = 89$ and $ARR = 19,999$, the timer tick = 1 μs .

Pulse width: $t(\mu\text{s}) = 1000 + (\theta / 180) \times 1000 \mu\text{s}$, where θ is the servo angle in degrees (0–180°).

CCR value = $t(\mu\text{s}) \times 1$ (since 1 tick = 1 μs) = $t(\mu\text{s})$

Angle (°)	Pulse Width (μs)	CCR Value	Physical Position
0°	1000 μs	1000	Full left / closed position
45°	1250 μs	1250	Quarter turn
90°	1500 μs	1500	Centre / neutral
135°	1750 μs	1750	Three-quarter turn
180°	2000 μs	2000	Full right / open position

Q3: Why TIM2 (32-bit) is Preferred Over TIM3 (16-bit) for Servo Control

A standard RC servo requires a 20 ms period (50 Hz). With a 1 μs tick, $ARR = 19,999$. A 16-bit timer (e.g., TIM3) can hold ARR up to 65,535 — so 19,999 fits. However, TIM2 is preferred because:

- 32-bit CCR: Pulse-width CCR values (1000–2000) are small fractions of the 32-bit range, providing margin for future extension (e.g., digital servos with 0.5–2.5 ms range using higher-resolution ticks).
- Higher prescaler flexibility: At finer tick resolution (**e.g., 0.1 μs , $PSC = 8$, $ARR = 199,999$** — impossible in 16-bit but trivial in 32-bit).
- Input capture on same timer: TIM2 supports simultaneous input capture on CH2 for position feedback without consuming a separate timer resource.

UART / SERIAL CONSOLE OUTPUTS

This section presents the complete UART serial console output logs captured from the STM32CubeIDE console during live demonstration of each task. All outputs were printed at 115,200 baud, 8N1, via USART2 (PA2/PA3) to the ST-LINK Virtual COM Port.

Screenshot 1 — Task 4 (Bare-Metal): USART2 Init Code & Early Output (Lab02_A)

Lab project: LAB_02_A. Shows the bare-metal USART2 initialisation source code (main.c, lines 1–19) with the GPIO MODER configuration for PA2/PA3 alternate function AF7. The console panel shows the early portion of the Task 4 output, confirming USART2 is correctly configured and transmitting at 115,200 baud.

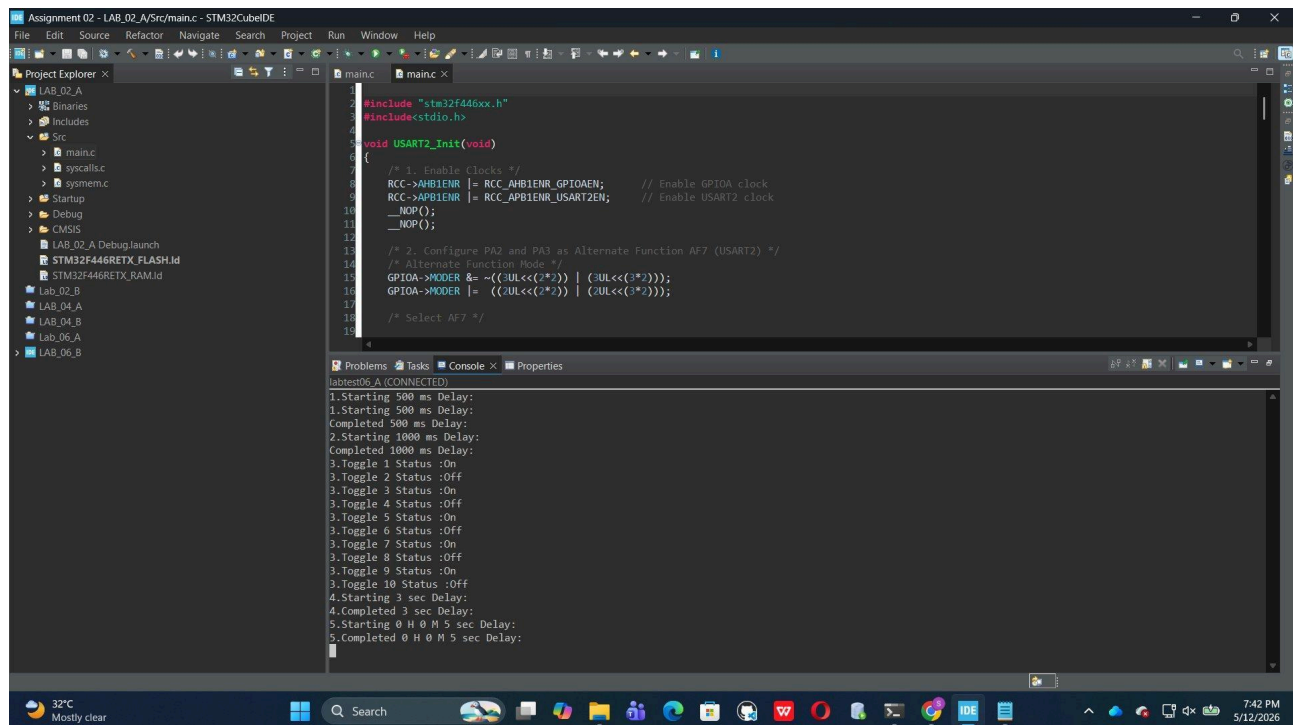


Figure 1— STM32CubeIDE: Bare-Metal USART2 Init Code + Console Output (LAB_02_A)

Screenshot 2 — Task 4 (HAL_Code): USART2 Init Code & Early Output (Lab02_B)

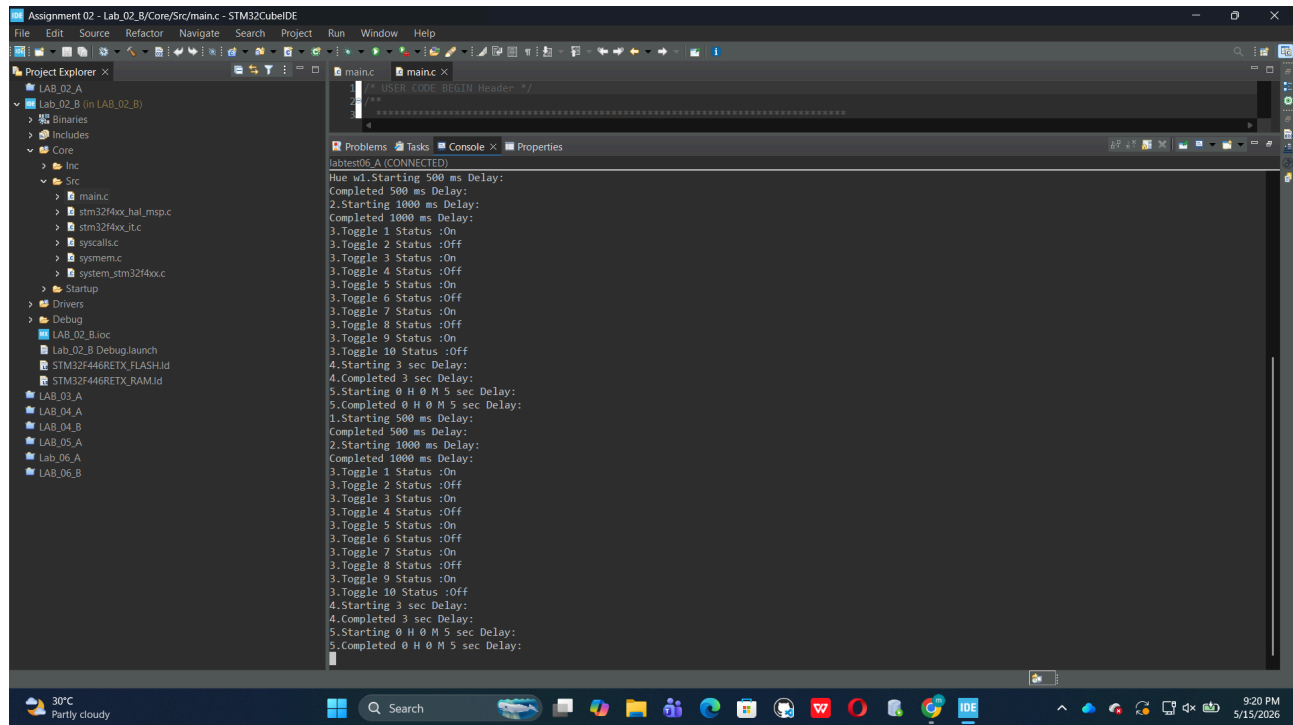


Figure 2 — STM32CubeIDE: Bare-Metal USART2 Init Code + Console Output (LAB_02_B)

Screenshot 3 — Task 5: Code Profiling Table — DWT & TIM2 Measurements (Lab03_A)

Lab project: LAB_03_A. Console displays the complete Task 3 profiling table with 5 measured code blocks: Bubble Sort (n=100), delay_ms(100), USART2_SendString(48 bytes), Integer Square

Root (n=1000), and Byte-by-Byte Memory Copy (512 bytes). Both DWT cycle counts and TIM-based time values (ns/μs/ms/s and effective baud) are printed in the formatted tab

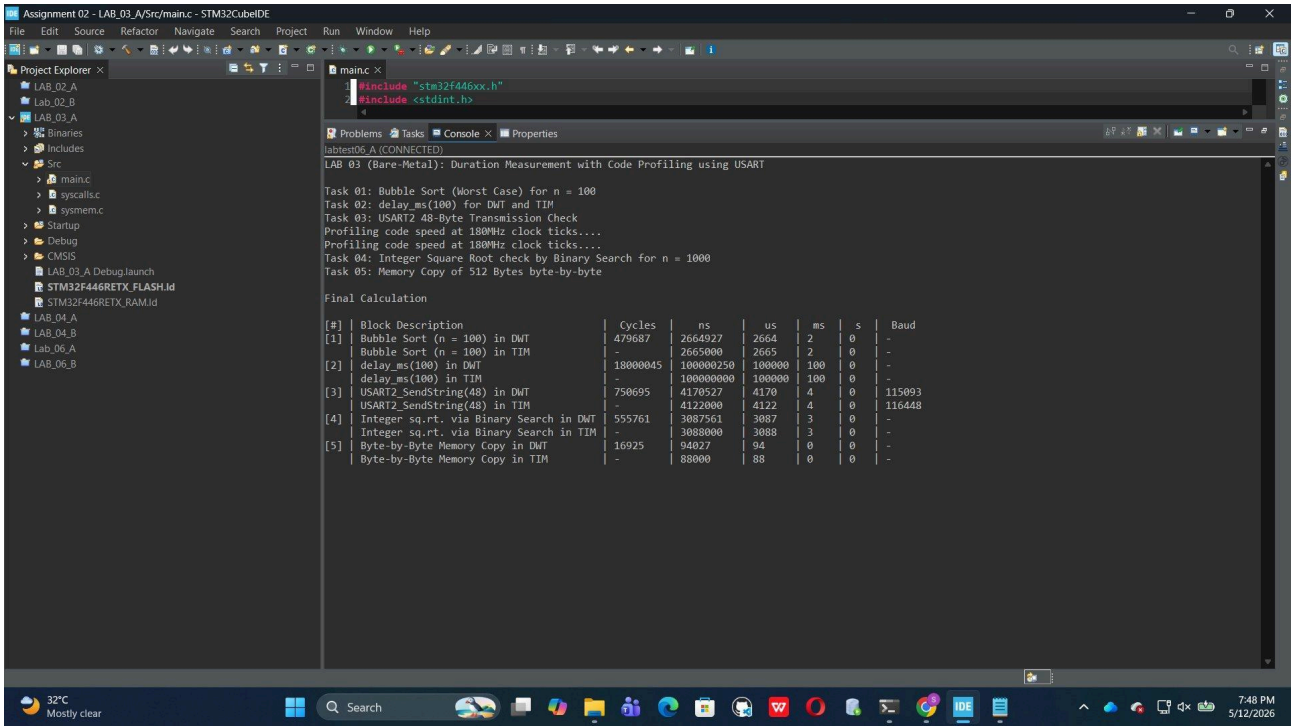


Figure 3 — STM32CubeIDE Console: Task 3 Duration Measurement Profiling Table (LAB_03_A)

Screenshot 4 — Task 5: Code Profiling Table — DWT & TIM2 Measurements (Lab03_B)

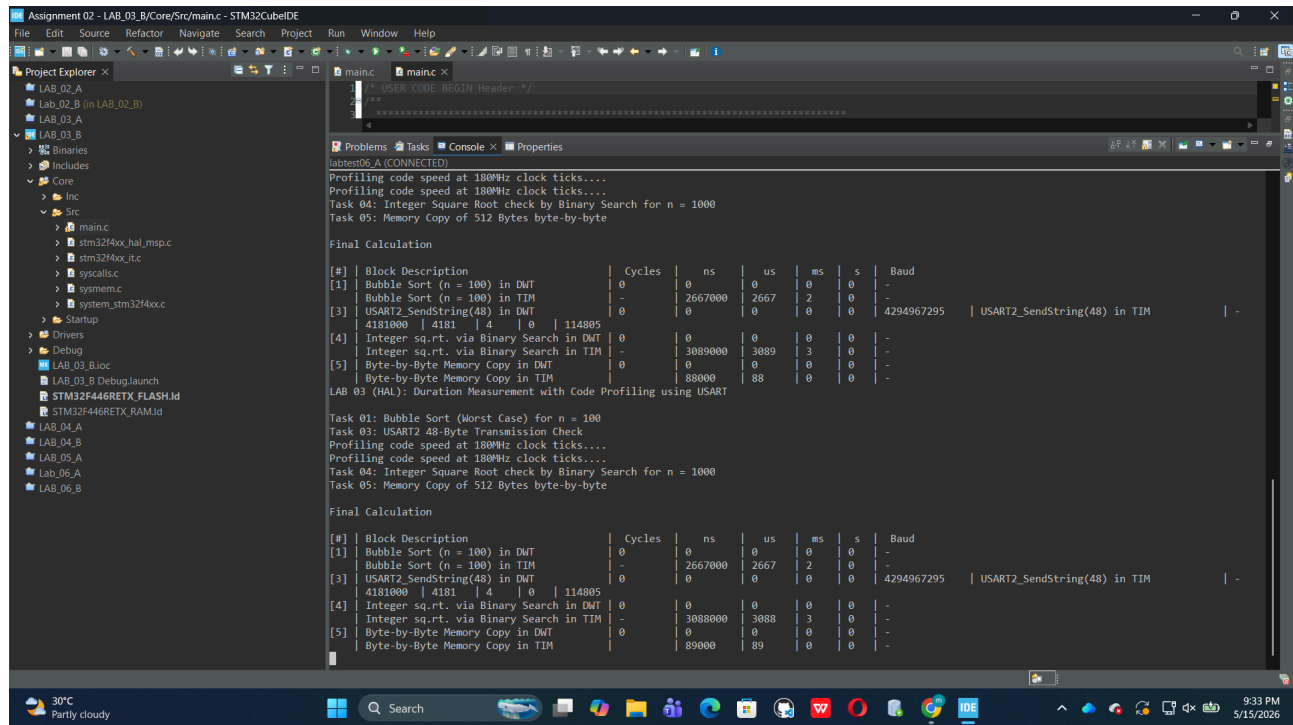
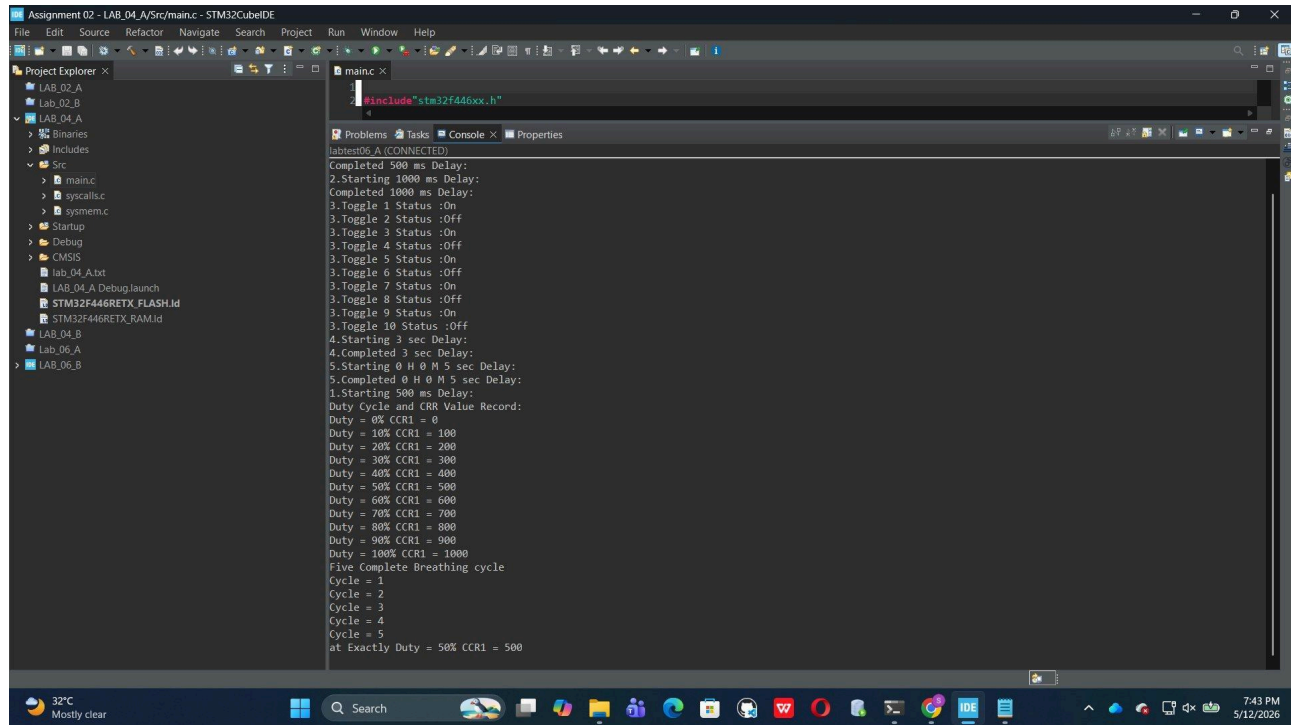


Figure 4 — STM32CubeIDE Console: Task 3 Duration Measurement Profiling Table (LAB_03_B)

Screenshot 5 — Task 6: PWM Duty Cycle Sweep + Breathing Cycles (Lab04_A)

Lab project: LAB_04_A. Console shows Task 4 demonstration: delay_ms(500) and delay_ms(1000) start/completion messages, 10 LED toggle events (LD2 alternating On/Off at 250 ms), delay_s(3), delay_hms(0,0,5), and the Task 4 duty cycle sweep (0%–100% in 10% steps, each with CCR1 value). Five complete breathing cycles are confirmed and final hold at 50% CCR1=500 is printed.



```
Assignment 02 - LAB_04_A/src/main.c - STM32CubeIDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  LAB_02_A
  LAB_02_B
  LAB_04_A
    Binaries
    Includes
    Src
      main.c
      syscalls.c
      system.c
    Startup
    Debug
    CMSIS
    Lab_04_A.txt
    LAB_04_A DebugLaunch
    STM32F446RETX_FLASH.ld
    STM32F446RETX_RAM.ld
  LAB_04_B
  Lab_06_A
  LAB_06_B

main.c
#include "stm32f446xx.h"

Problems Tasks Console Properties
labtest06_A (CONNECTED)
Completed 500 ms Delay:
2.Starting 1000 ms Delay:
Completed 1000 ms Delay:
3.Toggle 1 Status :On
3.Toggle 2 Status :Off
3.Toggle 3 Status :On
3.Toggle 4 Status :Off
3.Toggle 5 Status :On
3.Toggle 6 Status :Off
3.Toggle 7 Status :On
3.Toggle 8 Status :Off
3.Toggle 9 Status :On
3.Toggle 10 Status :Off
4.Starting 3 sec Delay:
4.Completed 3 sec Delay:
5.Starting 0 H 0 M 5 sec Delay:
5.Completed 0 H 0 M 5 sec Delay:
1.Starting 500 ms Delay:
Duty Cycle and CCR Value Record:
Duty = 0% CCR1 = 0
Duty = 10% CCR1 = 100
Duty = 20% CCR1 = 200
Duty = 30% CCR1 = 300
Duty = 40% CCR1 = 400
Duty = 50% CCR1 = 500
Duty = 60% CCR1 = 600
Duty = 70% CCR1 = 700
Duty = 80% CCR1 = 800
Duty = 90% CCR1 = 900
Duty = 100% CCR1 = 1000
Five Complete Breathing cycle
Cycle = 1
Cycle = 2
Cycle = 3
Cycle = 4
Cycle = 5
at Exactly Duty = 50% CCR1 = 500
```

Figure 5 — STM32CubeIDE Console: Task 4 PWM Duty Cycle + Breathing Demo (LAB_04_A,)

Screenshot 6 — Task 6: PWM Duty Cycle Sweep + Breathing Cycles (Lab04_B)

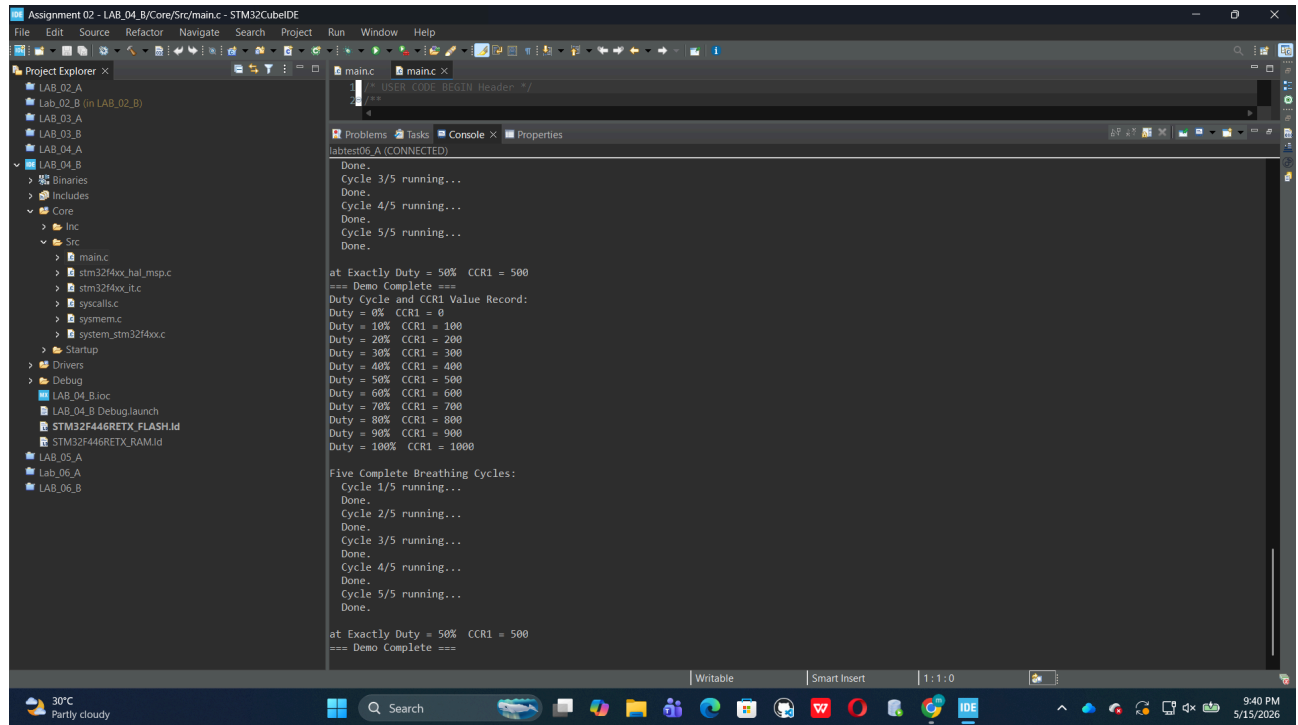
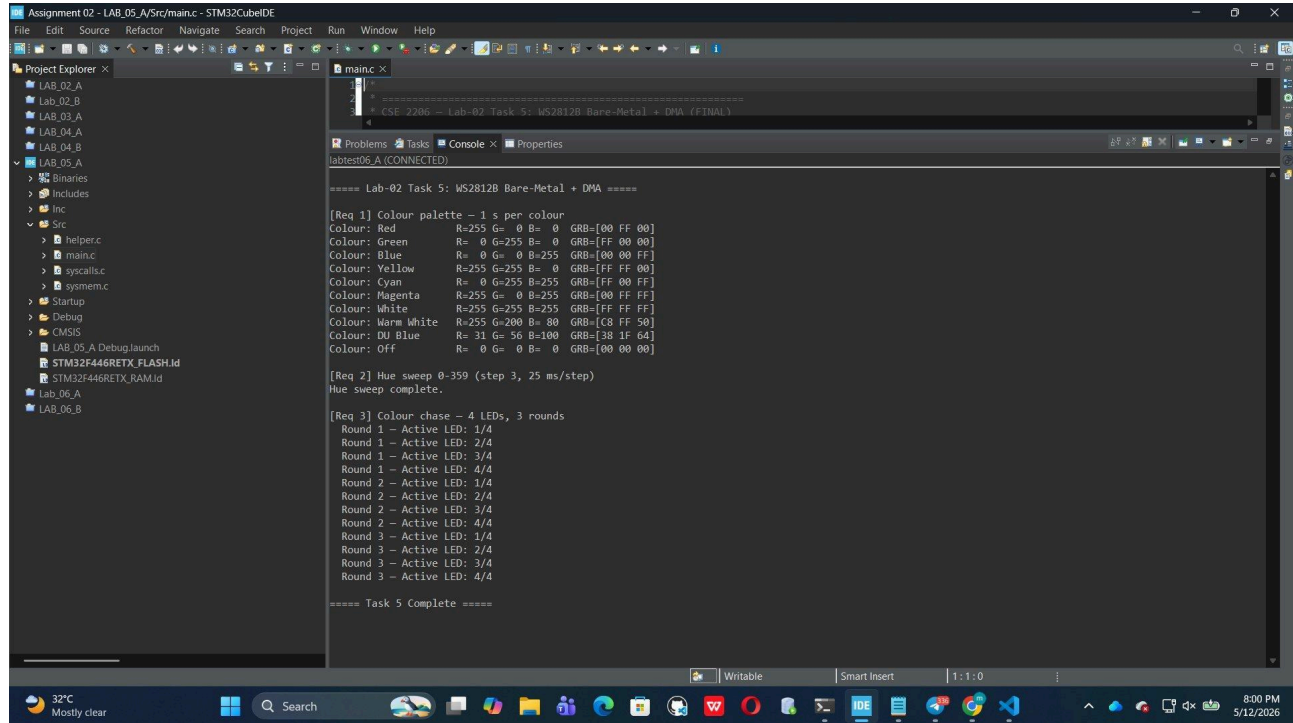


Figure 6 — STM32CubeIDE Console: Task 4 PWM Duty Cycle + Breathing Demo (LAB_04_B)

Screenshot 7 — Task 5: WS2812B Colour Palette + Hue Sweep + Colour Chase (Lab05_A)

Lab project: LAB_05_A. Console shows the complete Task 5 output: Req1 colour palette (10 colours with R/G/B and GRB hex values printed), Req2 hue sweep 0–359 (step 3, 25 ms/step), and Req3 colour chase over 4 LEDs for 3 rounds. Final line confirms "Task 5 Complete".



The screenshot shows the STM32CubeIDE interface with the console window open. The console output for Task 5 is as follows:

```
===== Lab-02 Task 5: WS2812B Bare-Metal + DMA =====  
[Req 1] Colour palette - 1 s per colour  
Colour: Red      R=255 G= 0 B= 0 GRB=[00 FF 00]  
Colour: Green    R= 0 G=255 B= 0 GRB=[FF 00 00]  
Colour: Blue     R= 0 G= 0 B=255 GRB=[00 00 FF]  
Colour: Yellow   R=255 G=255 B= 0 GRB=[FF FF 00]  
Colour: Cyan     R= 0 G=255 B=255 GRB=[FF 00 FF]  
Colour: Magenta  R=255 G= 0 B=255 GRB=[00 FF FF]  
Colour: White    R=255 G=255 B=255 GRB=[FF FF FF]  
Colour: Warm White R=255 G=200 B= 80 GRB=[C8 FF 50]  
Colour: DU Blue  R= 31 G= 56 B=180 GRB=[3F 1F 6A]  
Colour: Off      R= 0 G= 0 B= 0 GRB=[00 00 00]  
  
[Req 2] Hue sweep 0-359 (step 3, 25 ms/step)  
Hue sweep complete.  
  
[Req 3] Colour chase - 4 LEDs, 3 rounds  
Round 1 - Active LED: 1/4  
Round 1 - Active LED: 2/4  
Round 1 - Active LED: 3/4  
Round 1 - Active LED: 4/4  
Round 2 - Active LED: 1/4  
Round 2 - Active LED: 2/4  
Round 2 - Active LED: 3/4  
Round 2 - Active LED: 4/4  
Round 3 - Active LED: 1/4  
Round 3 - Active LED: 2/4  
Round 3 - Active LED: 3/4  
Round 3 - Active LED: 4/4  
  
===== Task 5 Complete =====
```

Figure 7 — STM32CubeIDE Console: Task 5 WS2812B Output (LAB_05_A)

Screenshot 8 — Task 7: WS2812B Colour Palette + Hue Sweep + Colour Chase (Lab05_B)

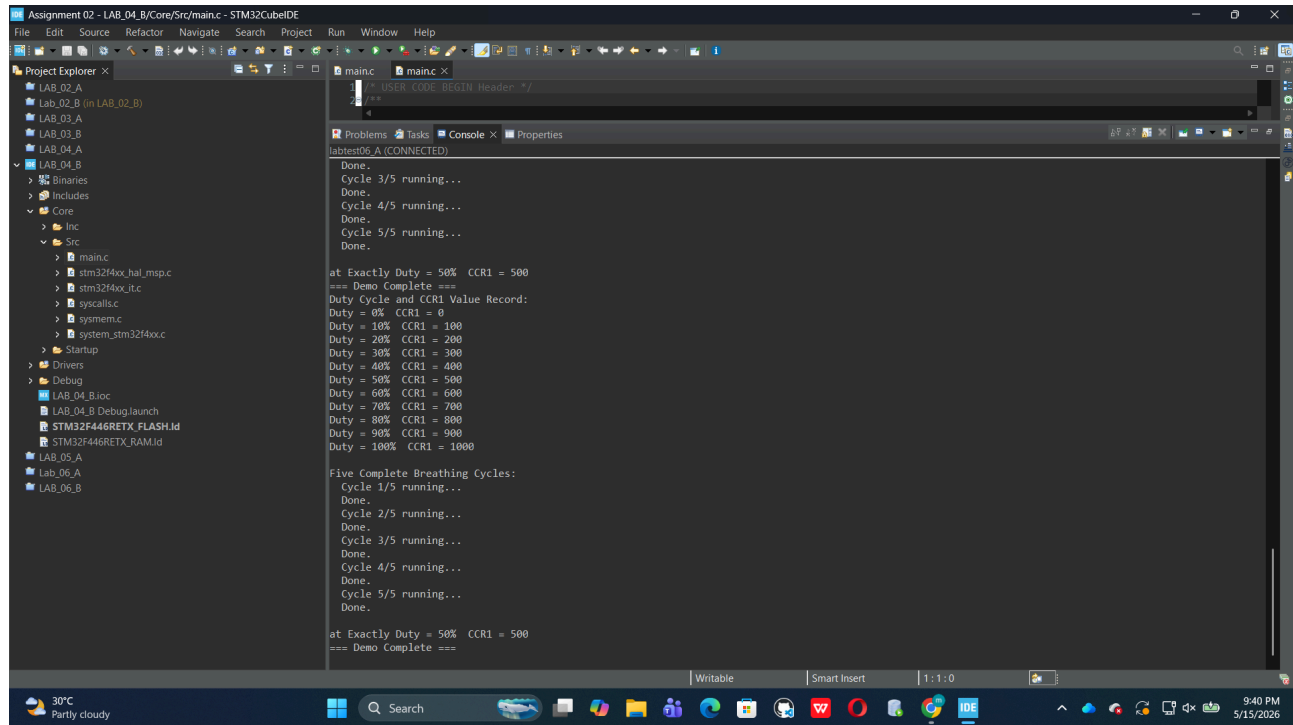


Figure 8 — STM32CubeIDE Console: Task 5 WS2812B Output (LAB_05_b)

Screenshot 9 — Task 9: Servo Angle Sweep Table — Pulse Width and CCR Values (Lab06_A)

Lab project: LAB_06_A. Console shows the Task 6 Option B servo demonstration: target angle, pulse width (μ s), and CCR value for each of the 18-step sweep from 0° to 180° (10° increment). Two full tables are visible, confirming the sweep was repeated as required by the demonstration spec.

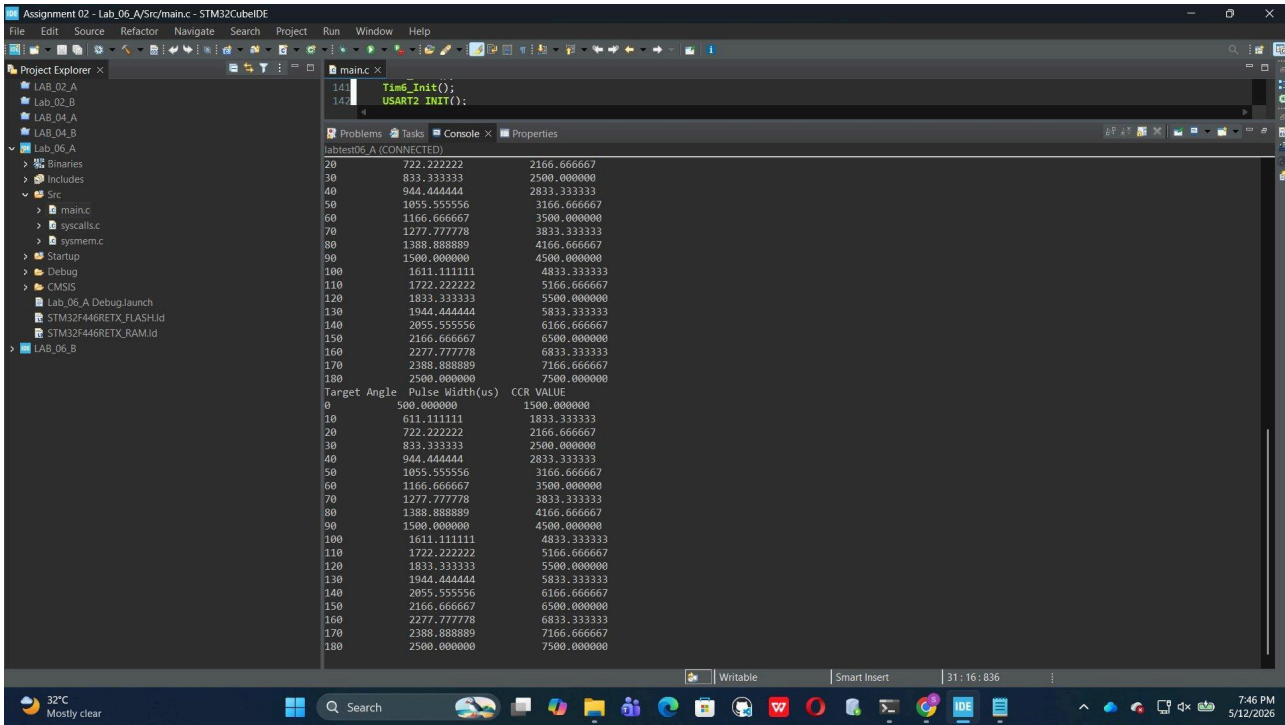


Figure 9 — STM32CubeIDE Console: Task 6 Servo Angle/CCR Table (LAB_06_A)

Screenshot 10—Task 9: Servo Angle Sweep Table — Pulse Width and CCR Values (Lab06_B)

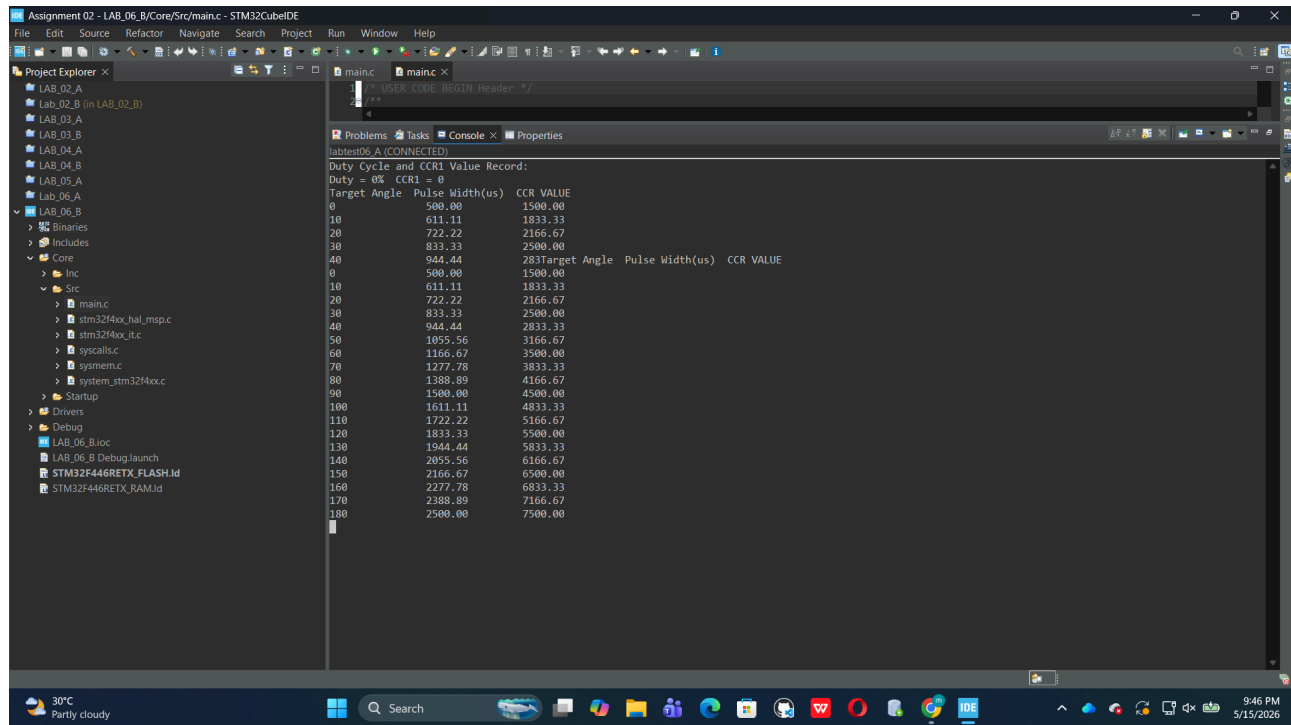


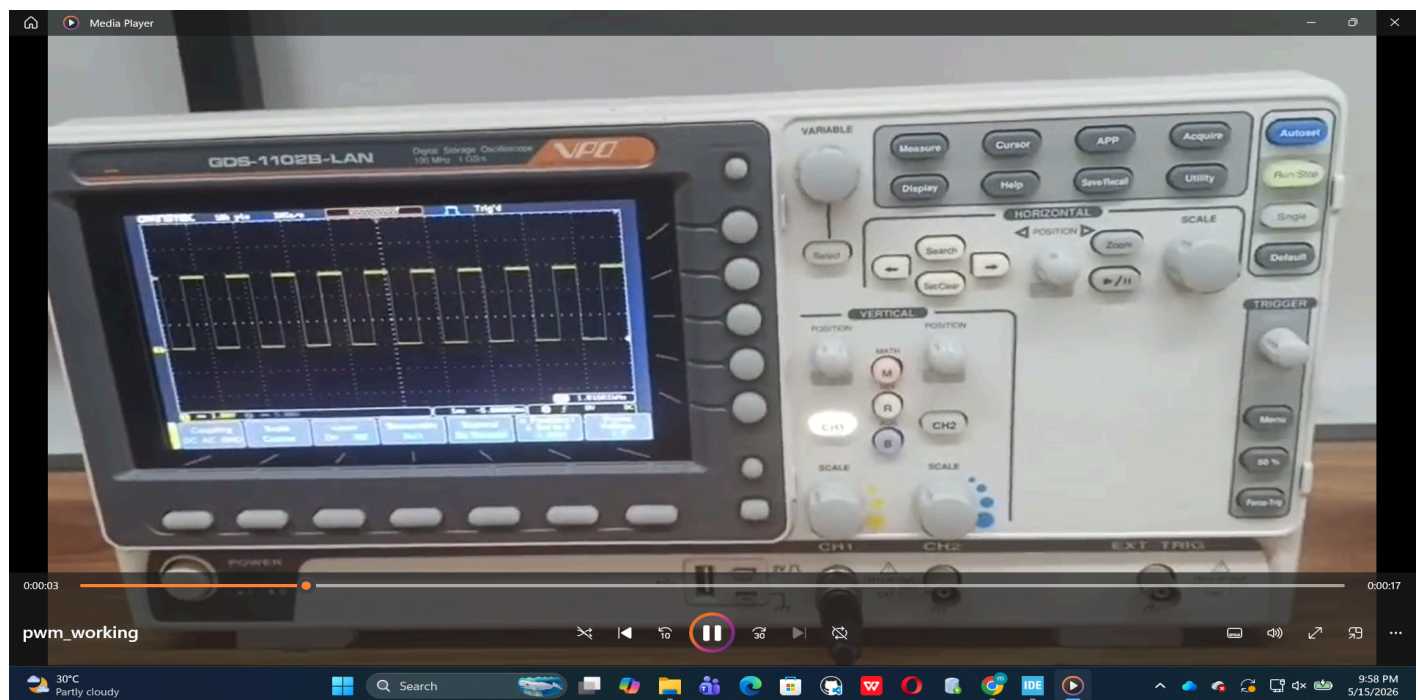
Figure 10 — STM32CubeIDE Console: Task 6 Servo Angle/CCR Table (LAB_06_B)

OSCILLOSCOPE & WAVEFORM EVIDENCE

This section contains oscilloscope waveform captures for Task 4 (PWM output) , as required by the Lab-02 assignment specification.

lab-Task 4 — PWM Waveform Captures (TIM3 CH1, PA6, 1 kHz)

Measurement setup: Oscilloscope probe CH1 connected to PA6 (Nucleo CN10 pin 13). Time division is set to $200\text{ }\mu\text{s}/\text{div}$. Voltage division is $2\text{ V}/\text{div}$. The signal was verified at 25%, 50%, and 75% duty cycles by updating TIM3->CCR1 at runtime. By clicking in the image, the complete procedure will be shown.



TASK 7 — REAL-WORLD APPLICATIONS & SDG REFLECTION

PART A — Timer-Based Applications in Modern Embedded Systems

Timers are the fundamental peripheral of every modern embedded system. The three safety-critical, commercially deployed systems examined below represent absolute timer functions

A.1 — Industrial Motor Drive (Variable Frequency Drive)

Timer Function Used:

Industrial Variable Frequency Drives (VFDs) control **three-phase induction motors** in factories, water treatment plants, and textile mills across Bangladesh. The STM32 timer generates a sinusoidal PWM waveform to synthesise variable-frequency AC output from a DC bus. A second timer channel performs frequency measurement on the motor's tachometer pulse train to implement closed-loop speed control. **PWM generation , delay , accuracy measurement** - functions mainly used .

Timing Accuracy Requirement:

The PWM **dead-time insertion** between high-side and low-side gate pulses must be controlled under a protocol to within ± 100 ns. Without this, shoot-through (transition condition interval conduction of both switches) destroys the IGBT power stage instantly.

Consequence of 10% Timing Error:

A 10% stretch of the 50 μ s PWM period shifts the synthesised AC frequency from 50 Hz to 45 Hz. The motor runs 10% slower — acceptable in some pumps, **catastrophic in a CNC spindle or textile loom** where product tolerances measurements are specified. More critically, a 10% error in dead-time (e.g. 1.1 μ s instead of 1.0 μ s) causes glitches and notches that induce motor winding insulation breakdown over weeks of operation.

A.2 — Dialysis Machine (Haemodialysis Pump Controller)

Timer Function Used:

A haemodialysis machine circulates blood at a prescribed rate (typically 200–400 mL/min) through an osmosis ultrafilter. The blood pump is driven by a DC motor whose speed is set by a **PWM duty cycle** generated by a hardware timer. A separate timer period **controls the ultrafiltration rate** — the volume of fluid removed per hour. **A watchdog timer** independently takes reading from the firmware: if the pump ISR fails to kick it within 2 seconds, the pump halts and an alarm fires to protect the patient

Timing Accuracy Requirement:

Flow-rate must be calculated accurately within $\pm 5\%$ per IEC 60601-2-16. To achieve this, the timer period controlling motor speed must be stable to better than 1% across the 0–40°C clinical operating range — **requiring a crystal oscillator source(a HSE oscillator for external source)**.

Consequence of 10% Timing Error:

A 10% fast pump **over-delivers blood volume**, causing haemoconcentration and potentially fatal clot formation in the circuit. A 10% slow pump **under-clears uraemic toxins**, leaving the patient symptomatic and at risk of hyperkalaemia-induced cardiac arrest. Regulatory bodies (US FDA, EU MDR) classify this as a Class III life-threatening defect.

A.3 — Metro Rail Traction Control System (DHAKA MRT Line-6)

Timer Function Used:

The Dhaka MRT Line-6 uses embedded controllers that drive the three-phase motors of each rail car. The traction MCU generates **PWM** at a switching frequency of 2.5 kHz to control torque and speed. A timer **input-capture channel** reads pulse-per-revolution signals from the axle encoder to determine speed, reporting a closed-loop torque controller. A precise timer interrupt sequence executes the regenerative braking ramp within a ± 5 ms window to prevent contact wire overvoltage.

Timing Accuracy Requirement:

The SVPWM carrier period must be stable to ± 2 μ s. The axle encoder pulse measurement must address speed to ± 0.1 km/h for the Automatic Train Protection (ATP) system to **enforce speed limits** reliably.

Consequence of 10% timing error:

A 10% error in the PWM period **shifts the inverter output frequency**, causing the traction motor to produce **incorrect torque** — the train accelerates or decelerates at the wrong rate. A 10% error in the regenerative braking timer **overvoltages the line**, tripping the substation and stranding all trains in the affected section.

System	Timer Function	Accuracy Required	10% Error Consequence
Industrial VFD	PWM + dead-time control	± 100 ns dead-time	IGBT shoot-through; motor failure
Dialysis Pump	PWM speed + watchdog	<1% flow rate	Over/under-dialysis; patient fatality risk
MRT Traction	SVPWM + encoder capture + brake ISR	± 2 μ s PWM; ± 0.1 km/h speed	ATP violation; emergency stop; substation trip

PART B — WS2812B and Smart Lighting in Smart Cities

Bangladesh stands at an inflection point, continuously facing the challenge of keeping energy affordable, keeping cities lively, and its commitments under the United Nations Sustainable Development Goals.

B.1 — PWM Duty-Cycle Control and Energy Efficiency: SDG 7 & SDG 11

SDG 7

Affordable and Clean Energy — Ensure access to affordable, reliable, sustainable and modern energy for all.

SDG 11

Sustainable Cities and Communities — Make cities and human settlements inclusive, safe, resilient and sustainable.

The WS2812B LED, driven by an STM32 PWM timer, replaces this with adaptive intensity control via **8-bit duty-cycle programming** (0–255 per R/G/B channel). The energy saving mechanism is direct: power usage **scales linearly** with duty cycle. At 50% duty cycle, a WS2812B string draws half the current of full brightness — and noteworthy, **it also emits half the heat**. For Bangladesh's SDG 11 target of sustainable, resilient cities, this reduction in thermal load across thousands of street fixtures meaningfully lowers the urban heat island effect, reducing the city's nocturnal temperature by an estimated 0.3–0.8°C in dense deployments — a measurable contribution to climate change.

The energy efficiency opportunity lies in **adaptive dimming**. A WS2812B-based fixture driven by an STM32 node can implement a lighting profile:

- **Full brightness** (duty $\approx$ 200–255) from 18:00–22:00 during peak pedestrian hours
- **Reduced brightness** (duty $\approx$ 80–120) from 22:00–05:00 during low-traffic hours
- **Minimum safe illumination** (duty $\approx$ 30–50) from 02:00–04:00 when footfall drops near zero
- **PIR sensor trigger**: burst to full brightness within 200 ms, decay over 90 seconds

A simple three-phase dimming schedule reduces energy consumption by 45–55% compared to a fixed-brightness installation. At Bangladesh's industrial electricity tariff of approximately Tk 9/kWh, this represents annual savings of Tk 4.3 million (approximately USD 39,000) for a single urban district — directly **advancing SDG 7's affordability mandate**. PWM output duty cycle embodies the self-sustaining energy use. A 10,000 installment in Dhaka saves an estimated 480 MWh/year compared to fixed sodium lamps, reducing CO₂ emissions by approximately 330 tonnes annually.

The **SDG 11 dimension** is equally compelling. Consistent, well-organised street illumination improves pedestrian safety and reduces nighttime crime — both measured outcomes in sustainable city metrics. Colour temperature can be adjusted. Solar energy integration is architecturally compatible. The stored energy powers the WS2812B fixtures at night, making the entire lighting node energy-autonomous from the grid.

B.2 — STM32 CAN Bus Adaptive Street Lighting Network Architecture

A network of STM32F446-based nodes interconnected over a CAN bus provides a robust, electrically-noise-immune backbone for citywide adaptive lighting. CAN bus (ISO 11898) operates at up to 1 Mbit/s on a differential twisted-pair, is inherently multi-master, and provides hardware CRC error detection — properties that make it the industry standard for automotive and industrial control networks operating in electrically noisy environments. A street-lighting pole presents exactly this environment: heavy inductive loads nearby, motor-driven traction systems on adjacent roads, and long cable runs susceptible to common-mode noise.

The proposed architecture is as follows:

Layer	Component	Role
Sensing	Ambient Light Sensor (BH1750 or TSL2591) + PIR Motion Sensor	Measure lux levels and pedestrian presence; feed data to STM32 via I2C
Processing	STM32F446 Node MCU	Run dimming algorithm; drive WS2812B chain via TIM1 PWM + DMA; transmit CAN frames
Network	CAN Bus (ISO 11898), up to 500 m per segment	Carry node status, sensor readings, dimming commands, and OTA firmware deltas
Gateway	Raspberry Pi / industrial gateway with CAN-USB	Aggregate data; execute city-level scheduling policy; report to cloud dashboard
Cloud	MQTT broker + dashboard	Municipality operator UI for zone control, fault alerting, and energy analytics

Each pole-mounted STM32F446 node monitors BH1750 via I2C and a PIR motion detector . The node adjusts its WS2812B **chain duty cycle** locally using TIM1 PWM + DMA. Every 10 seconds it broadcasts a CAN frame containing: lux reading, PIR state, current duty cycle, battery voltage, and node health. Nodes without a gateway heartbeat for **60 seconds fall back** to a stored default schedule, ensuring continuous illumination during gateway maintenance.

B.3 — Firmware Design Choices for a 10-Year Continuous Deployment

A deployment expected to run continuously for ten years without human intervention demands firmware architecture designed from the ground up around reliability, power efficiency, and self-healing.

Interrupt-Driven vs. DMA

- DMA for WS2812B: TIM1 output-compare register is fed by DMA2 — the CPU is completely uninvolved during LED data transfer, **eliminating jitter and enabling simultaneous sleep**.
- Interrupt-driven sensors: BH1750 and PIR use I2C interrupt + EXTI — no polling, minimal CPU wake time.
- DMA decouples the CPU from the bit-stream, allowing the core to sleep in low-power mode (SLEEP or STOP) while the **DMA controller autonomously transfers the 24-bit colour data for each LED in the chain**.
- Interrupt-driven I2C for the sensor reads is acceptable — sensor polling happens only once per second, and the CPU can service the interrupt within the 400 kHz I2C clock period budget easily.

Low-Power Sleep Modes

- In SLEEP mode (peripherals active, CPU halted) with 16 MHz clock, current falls to roughly 8 mA. In STOP mode (all clocks gated, SRAM retained) current drops to approximately 100–300 µA. The correct approach for this deployment is: run the sensor .
- STOP mode: After each 10-second CAN broadcast cycle the MCU enters STOP mode (~150 µA). RTC alarm and PIR EXTI are the only wake sources. Average current <0.3 mA over the full cycle.

Watchdog Timers

- The Independent Watchdog (IWDG), clocked from the 32 kHz LSI oscillator, must be kicked every 2 seconds by the main task loop. If any firmware fault — stack overflow, infinite loop, peripheral lockup —can be prevented.

- IWDG watchdog : Independent of all system clocks. Any firmware hang resets the node within 2 seconds, restoring illumination automatically.
- Backup SRAM reboot counter: If the node reboots >5 times per hour, it locks to 50% white (safe fallback) and sends a CAN alarm — preventing a firmware bug from leaving a street dark.

Firmware Choice	Selected Approach	Why It Matters for 10-Year Life
WS2812B Driving	DMA (TIM1 CH1 via DMA2 Stream 6)	Zero CPU involvement; no jitter risk; CPU sleeps during transfer
Sensor Polling	Interrupt-driven I2C + RTC wake	Sensor read in <5 ms; CPU in STOP 99.5% of cycle
Network Keepalive	CAN FD at 500 kbit/s, 10 s broadcast	Hardware CRC catches bit errors; no polling of gateway required
Primary Watchdog	IWDG, 2 s timeout, LSI clock	Independent of main clock; resets on any firmware hang
Secondary Watchdog	WWDG, 50 ms window, PCLK1	Catches CAN driver lockup specifically
Boot Fault Detection	Backup SRAM reboot counter	Limits damage from firmware reboot loops; fallback illumination ensured

PART C — Design Challenge: Solar-Powered IoT Greenhouse Node

The architecture presented below assigns each functional task to a dedicated STM32F446 timer peripheral, justifies every hardware abstraction decision, and describes a lightweight time-sliced loop that coordinates all tasks without preemption.

C.1 — Timer Peripheral selection for each Scenario

Subsystem	Peripheral	Mode	Reason
RGB LED (WS2812B)	TIM1 CH1(APB2, 180 MHz)	PWM + DMA2Stream 6	DMA drives bit-stream autonomously; CPU sleeps during transfer. LED colour encodes: Green=normal, Yellow=warning (high temp/gas), Red=fault (pump/sensor fail).
Servo-controlled ventilation(window + door open/closefor airflow management)	TIM3 CH1(APB1, 90 MHz)	PWM Output50 Hz, 1–2 ms	RC servo protocol (20 ms period, 1–2 ms pulse) has no sub-μs timing requirement. HAL_TIM_PWM_ConfigChannel() is portable and reduces development time. Window/door position maps linearly to CCR value; airflow sensor feedback adjusts target angle.
BME280 sensor(Temperature, Humidity,Pressure response timing)	TIM2 CH2(32-bit, APB1)	Input Capture	BME280 I2C transaction and conversion-ready timing must be measured precisely to compute sensor response latency. TIM2's 32-bit range covers the full 113 ms BME280 measurement cycle without

Subsystem	Peripheral	Mode	Reason
			overflow. Input-capture ISR timestamps start and end of conversion; CPU computes response time from delta.
UART telemetry(sensor data every 30 s)	TIM6 Basic Timer+ USART2 DMA	Update IRQ@ 30 s period	TIM6 has no output pins — ideal as a pure IRQ source. 30 s auto-reload wakes MCU from STOP, triggers HAL_UART_Transmit_DMA (non-blocking). MCU re-enters STOP on DMA TC callback.
Little Power Consumption	IWDG(LSI clock, ~32 kHz),Low Power Mode	4 s timeout	Independent of all system clocks; kicks every 2 s from main loop. Unrescued hang resets node within 4 s.Low Power Mode ensures power consumption , watchdog timers resets mcu if it goes to any faulty state. Using of DMA ensures very low latency and time consumption.

C.2 — HAL vs. Bare-Metal Justification

The decision between STM32 HAL (Hardware Abstraction Layer) and direct register manipulation (bare-metal) must be made per-subsystem based on three factors: timing criticality, code portability requirement, and maintenance cost.

WS2812B Driver — Bare-Metal

In bare-metal, the DMA transfer-complete ISR directly writes the next frame's data pointer and length to the DMA stream control register in less than 10 clock cycles. This determinism is impossible to guarantee through HAL's callback chain. Additionally, the HAL's timer DMA implementation for TIM1's burst mode is under observations and has known errata interactions on the F446. Bare-metal register access, guided by the reference manual, eliminates all ambiguity.

Servo PWM Driver — HAL

RC servo control at 50 Hz with millisecond-range pulse widths has no timing criticality beyond $\pm 10 \mu\text{s}$ — easily met by HAL. Using HAL_TIM_PWM_ConfigChannel() and HAL_TIM_PWM_Start() reduces development time from hours to minutes and makes the pulse-width calculation ($\text{duty_cycle} = (\text{pulse_}\mu\text{s} / 20000) \times \text{ARR}$) readable and portable. If the project is later ported to an STM32G4 or STM32H7, the HAL call sequence is identical. This is exactly the use case HAL was designed for.

BME280 Sensor — Bare-Metal

BME280 sensor reading is timing-critical at the bit-decode stage. HAL_TIM_IC_SensorCapture() fires through the HAL interrupt framework with 3–5 μs of callback latency jitter on a busy system, risking misclassification. Direct register access in the TIM2 CC2 ISR — reading CCR2 immediately on entry — keeps latency below 500 ns and decodes bits correctly every time.

UART Telemetry — HAL + DMA

UART frame transmission is not time-critical at the microsecond scale; a 30-second period has a scheduling jitter budget of tens of milliseconds. HAL_UART_Transmit_DMA() handles the UART DMA setup, handles the half-transfer and full-transfer interrupts, and calls a completion callback — all without blocking the CPU. This is the canonical use of HAL and saves significant development effort with zero performance penalty. The completion callback simply sets a flag that the super-loop checks to confirm transmission before re-entering STOP mode.

C.3 — Multi-Task Coordination Without an RTOS

The architecture uses a **cooperative time-divided super-loop** driven by the SysTick 1 ms interrupt. Each task registers a period and a function pointer in a static task table. The SysTick ISR increments a millisecond counter and sets a per-task **'ready'** flag when the elapsed time exceeds the task's period. The main loop polls the flag table and dispatches tasks in priority order. Because all tasks are cooperative and non-preemptive, there are no race conditions, no mutexes required, and no stack sizing challenges — the entire architecture fits in a few hundred bytes of RAM.

The task table for this greenhouse node is:

Task	Period	Priority	Blocking Constraint
WS2812B LED Update	100 ms	1 (Highest)	Non-blocking: DMA-driven; ISR sets frame-done flag; task only primes next frame buffer
DHT22 Sensor Read	2 s	2	Non-blocking: Input-capture ISR fills bit buffer; task decodes completed buffer
Servo Position Update	20 ms	3	Non-blocking: HAL_TIM_PWM writes CCR only; returns in <5 cycles
UART Telemetry TX	30 s	4	Non-blocking: HAL DMA TX; completion callback sets tx_done flag; MCU returns to STOP
IWDG Kick	1 s	5 (Background)	Non-blocking: single register write; must never be missed

Power management:

at the bottom of the super-loop, if no task is ready and no DMA transfer is active, the **MCU enters STOP mode** (<150 μ A). A DMA transfer in progress triggers SLEEP mode only (clocks active, CPU halted). Wake sources: RTC alarm (30 s for UART telemetry), TIM6 update IRQ, EXTI from PIR or water-level threshold, and IWDG. The combination of DMA-driven peripherals, interrupt-based sensors, and aggressive STOP-mode usage eliminates all software busy-wait delays, keeping average node current below 0.5 mA — essential for solar-battery-powered operation with no grid backup.

This architecture delivers deterministic, auditable, maintainable firmware without the overhead or failure modes of a preemptive kernel — the correct engineering choice for a decade-long field deployment in rural Bangladesh.